

SLIDE 02 · NỖI ĐAU & SỐ LIỆU (45 GIÂY)

Thiệt hại kép: Lãng phí 30–60 phút mỗi buổi LAB

Khởi động chậm + Sửa lỗi phân công giữa chừng = "Đốt" mất 25% – 33% tổng thời gian thực hành.

 Khởi động chậm ban đầu

55.6%

Nhóm mất từ 15 đến >30 phút chỉ để thống nhất đề bài

 Sửa sai & Chia lại task

75.0%

Nhóm mất thêm 5 đến >30 phút vì việc phân chia lỗi

 Giảm chất lượng bài nộp

63.9%

Nhóm bị giảm chất lượng bài & 41.7% trễ hạn nộp

Phân bổ thời gian lãng phí ở 2 giai đoạn

GIẢI ĐOẠN 1: THỐNG NHẤT BAN ĐẦU (Q3, n = 36)

5 đến 15 phút (Chấp nhận được) 41.7% (15 nhóm)

15 đến 30 phút (Bắt đầu lãng phí) 36.1% (13 nhóm)

Trên 30 phút + Bế tắc hoàn toàn 19.5% (7 nhóm)

GIẢI ĐOẠN 2: SỬA ĐỔI & CHIA LẠI TASK GIỮA CHỪNG (Q5, n = 36)

Mất thêm 5 đến 15 phút sửa việc 44.4% (16 nhóm)

Mất thêm 15 đến >30 phút sửa việc 30.6% (11 nhóm)

 Tổng thời gian lãng phí: 30–60 phút / 180 phút thực hành (~25–33% thời lượng mỗi buổi).

Phân tích Tương quan: Phương pháp vs Hậu quả

Cách nhóm chia việc	Tỷ lệ nhóm	Tỷ lệ sót task	Tỷ lệ trùng code
Tự nhận việc	36.1% (13)	69.2%	46.2%
Leader phân chia	33.3% (12)	41.7%	41.7%
Cùng thảo luận	25.0% (9)	55.6%	33.3%

 Có phần việc bị bỏ sót (Checklist nộp) 55.6%

 Có người chưa biết làm gì trong buổi 50.0%

 Hai hoặc nhiều bạn làm trùng một task 41.7%

 Khó theo dõi tiến độ nhóm đã xong đến đâu 38.9%

 86.1% nhóm đọc README nhưng vẫn mù mờ vì tài liệu dài dòng, không phân rã thành các đầu việc nguyên tử (atomic tasks).

SLIDE 03 · VÌ SAO CHỌN TÍNH NĂNG NÀY (45 GIÂY)

Bảng Impact: Cân nhắc 3 ứng viên tính năng

Chọn giải pháp có bằng chứng mạnh nhất, giải quyết trực tiếp 30–60 phút lãng phí của người dùng.

Ứng viên tính năng	Số người gặp (Evidence)	Tần suất	Thiệt hại mỗi lần	Build 24h?	Quyết định
1. Bot Q&A giải đáp code Hỏi đáp cú pháp trong giờ LAB	14/36 người (38.9%)	Thường xuyên	5–10 phút/lần	Khả thi	❌ LOẠI Đã có ChatGPT/VLearn tutor
2. Auto-Grader chấm điểm Tự chấm testcase trước giờ nộp	18/36 người (50.0%)	Cuối buổi LAB	Mất điểm rubric	Khó kịp	❌ LOẠI Phụ thuộc hệ thống BTC
3. LABSTART: Repo-to-DAG Pilot Bóc tách repo + DAG + Checklist nộp	36/36 người (100%)	Mỗi bài LAB	30–60 phút/buổi	Khả thi cao	✓ CHỌN DUY NHẤT

1. Mức Automation (Augment)

Hỏi trước hay làm luôn? Nhóm chọn cơ chế **Augmentation**: AI chỉ đề xuất bản nháp, con người giữ quyền quyết định cuối cùng.

- **Bước 1:** AI parse repo sinh sơ đồ DAG gợi ý.
- **Bước 2:** Đội trưởng kéo thả phân công lại.
- **Bước 3:** Học viên nhận và xác nhận scope.
- **Cam kết:** Không tự động áp đặt task cứng nhắc.

✓ Giữ quyền kiểm soát cho Leader

2. Chi phí Sai sót (Cost of Error)

Bỏ sót task đắt hơn nhiều so với **chia dư task**. Hệ thống được tối ưu hóa đặc biệt cho chỉ số **High Recall (≥95%)**.

- **Sót checklist:** Mất điểm rubric bài LAB.
- **Chia nhỏ Atomic Tasks:** Dễ kiểm thử & debug.
- **Gán 1 owner / file:** 0% xung đột git merge.
- **Chế độ kiểm tra chéo:** Tự động tạo test checklist.

✓ High Recall ≥95% cho Rubric

3. Human-in-the-loop & HAX G10

Cẩn trọng khi AI không chắc chắn (HAX G10). Cho phép xác thực lại các giả định trước khi kích hoạt quy trình làm bài.

- **Đề thiếu rubric:** Không đoán bừa, gắn cờ đỏ.
- **Tự sinh câu hỏi:** Gợi ý Leader hỏi TA ngay.
- **One-click Sync:** Đẩy task sang Discord nhóm.
- **Audit Log:** Lưu lịch sử phân công minh bạch.

✓ HAX G10: Minh bạch khi không chắc

SLIDE 04 · GIẢI PHÁP & DEMO LÁT CẮT (2 PHÚT)

Lát cắt: Repo/README → DAG Task + Checklist Nộp



Thiết kế theo nguyên tắc HAX/PAIR: Augment con người, không làm thay việc ra quyết định.

⚙️ Kiến trúc 4 lớp của Sản phẩm

LỚP 1 · INGESTION: Parse README & Repo

Quét cây thư mục, trích xuất mục tiêu, điều kiện nộp bài và tiêu chí rubric chấm.

LỚP 2 · REASONING: Gemini 1.5 Structured Output

Bóc tách thành các đầu việc nguyên tử (Atomic Tasks) kèm JSON Schema kiểm định nghiêm ngặt.

LỚP 3 · DAG ENGINE: Sinh đồ thị phụ thuộc & Phân quyền File

Chỉ rõ task song song vs tuần tự; gắn nhãn file sở hữu để triệt tiêu xung đột git.

LỚP 4 · ACTIONABLE UI: Checklist Nghiệm thu & Match Kỹ năng

Gợi ý phân công theo hồ sơ cá nhân (Data, Prompt, Code, Eval) + Checklist trước khi nộp.

Repo URL → AST Parser → Gemini Flash → DAG Engine → Checklist & Kanban

Model: Gemini 1.5 Flash (Latency: ~42s, \$0.003/run)

STRUCTURED JSON

🎬 2 Kịch bản Demo Trực tiếp (Live Demo)

Case 1: Chuẩn (Happy Path)

Hoàn hảo

Đưa vào repo Hackathon Day 05 → AI sinh 5 module công việc song song, xác định rõ file nào do ai phụ trách trong **42 giây**, 0% trùng lặp.

Case 2: Chỗ khó (Ambiguous / Failure Path)

Xử lý thông minh

Đề bài không ghi rõ định dạng nộp file → AI **không tự bịa** (theo HAX G10), lập tức gắn cờ cảnh báo *"Thiếu tiêu chí nộp bài"* và sinh sẵn câu hỏi để đội trưởng hỏi TA.

4 ĐƯỜNG TRẢI NGHIỆM ĐÃ KIỂM THỬ ĐẦY ĐỦ (SPEC §4):

● Happy Path: Repo đủ thông tin (42s)

● Low-confidence: Đề bài mơ hồ (Cảnh báo)

● Failure Path: Thiếu rubric (Gợi ý hỏi TA)

● Correction Path: Leader kéo thả sửa

⚡ **Sẵn sàng demo live:** Đội trưởng mở tab VLearn Pilot, dán link repo và quan sát kết quả trực tiếp trong 2 phút thuyết trình.

SLIDE 05 · KẾT QUẢ ĐO & QUALITY BAR (45 GIÂY)

Đo lường khoa học: Vượt Quality Bar cam kết

Kiểm nghiệm trên Golden Dataset 15 bài LAB và bộ phản hồi của 9 Early Adopters sẵn sàng thử nghiệm.

📋 Task Recall (Độ phủ)

96.2%

Cam kết Quality Bar: $\geq 95.0\%$

✓ Không sót checklist nộp trên 15 bài LAB

⚡ Tốc độ phân tích

42s

Cam kết Quality Bar: ≤ 120 giây

✓ Nhanh gấp 30 lần việc họp bàn thủ công

💛 Willing Users

25.0%

9 bạn đăng ký test thử MVP ngay

✓ 4 bạn để lại Discord & Zalo trực tiếp

📊 Đối chiếu Quality Bar chốt từ CP4 (Golden Dataset 15 bài LAB)

Tiêu chí chất lượng	Chỉ số cam kết	Thực tế đạt được	Trạng thái
Task Coverage	$\geq 95\%$	96.2%	✓ ĐẠT
Thời gian phản hồi	≤ 120 giây	42 giây	✓ VƯỢT
Trùng lặp file sở hữu	0% conflict	0% conflict	✓ ĐẠT
Trích xuất Rubric nộp	$\geq 90\%$	100%	✓ ĐẠT

🔗 Phương pháp: Kiểm chứng mù độc lập trên 15 repo bài LAB mẫu các module trước (Machine Learning, Deep Learning, Full-stack AI).

🔍 Key Failure Mode & Cách xử lý trong 24h

Lỗi 1: Notebook (.ipynb) quá nặng token metadata

Metadata cell JSON làm tăng token gấp 3 lần và kéo dài latency lên 85s. → Giải pháp: Thêm AST pre-processor chỉ trích cell Markdown & header hàm, giảm 65% token, latency về 42s.

Lỗi 2: Đề bài thiếu quy định file nộp cụ thể

AI có xu hướng tự đoán tên file nộp. → Giải pháp HAX G10: Ép cờ cảnh báo đỏ "Thiếu rubric", chặn đoán bừa và tự sinh câu hỏi để Leader hỏi ngay TA.

💛 Phản hồi Early Adopters: 9 bạn sẵn sàng dùng thử bản Alpha cho bài LAB kế tiếp!

SLIDE 06 · NẾU CÓ THÊM 1 TUẦN (30 GIÂY)

Từ Prototype đến Sản phẩm VLearn chính thức 🌟

Kế hoạch hành động tập trung giải quyết triệt để phản hồi từ các bài test thực tế.

🚀 3 Việc ưu tiên triển khai trong 1 tuần tới

- 1 Tích hợp trực tiếp Discord Bot VLearn**
Tự động tạo thread riêng cho từng task và bắn notification khi có bạn kệt việc.
- 2 Real-time Git Commit Watcher**
Theo dõi tiến độ nộp bài qua commit GitHub để cảnh báo sớm rủi ro trễ deadline trước 60p.
- 3 Auto Scaffolding Code Skeleton**
Tự động sinh file mẫu và test case rỗng để thành viên bắt tay vào làm ngay.

LỘ TRÌNH 7 NGÀY SPRINT:

- Ngày 1–3: Hoàn thiện Webhook Discord + GitHub Events API.
- Ngày 4–5: A/B Testing trên 3 nhóm thực tế từ 9 Early Adopters.
- Ngày 6–7: Đóng gói module VLearn và bàn giao tài liệu kỹ thuật.

🚀 **Tầm nhìn:** Trở thành module tiêu chuẩn "Lab Pilot" tích hợp trong nền tảng VLearn VinUni.

💡 Bài học lớn nhất của nhóm

🔥 VLearn Mindset

"AI trong sản phẩm không phải là làm thay con người, mà là dọn sạch sự mơ hồ và chi phí điều phối vô hình, để học viên dành trọn 100% thời gian cho tư duy và sáng tạo."

3 NGUYÊN TẮC SẢN PHẨM CỐT LỖI ĐÃ RÚT RA:

1. **Augmentation > Automation:** Trao quyền cho Đội trưởng duyệt và kiểm soát.
2. **High Recall > Precision:** Bỏ sót task checklist nguy hiểm hơn chia dư việc nhỏ.
3. **HAX G10:** Khi đề bài mơ hồ, AI phải biết cảnh báo thay vì suy đoán bừa.

ĐỘI NGŨ THỰC HIỆN · TEAM E403 – SLOPPERS:

👑 **H.P. Trọng**
Product Lead & Spec Architect

📊 **Thành viên 2**
Evidence & Survey Data Analysis

💻 **Thành viên 3**
Prototype & Model Calls

🎯 **Thành viên 4**
Eval Benchmark & Quality Bar

